

Bristle

Multi-source problem discovery for builders.

BUILD PLAN · VERSION 1.0

A Spec Driven Development plan organized as 45 vertical slices across 8 tiers.

Target timeline: 14 weeks to public launch.

Framework: GitHub Spec Kit with Claude Code.

Reference document for Claude Code

May 2026

How each slice works

For every slice, the workflow is identical and follows Spec Kit:

- **/speckit.specify** — write the slice spec with checkable acceptance criteria
- **/speckit.plan** — generate the technical plan, iterate with "don't implement yet" until clean
- **/speckit.tasks** — break into atomic tasks, one commit each
- **/speckit.implement** — execute tasks one at a time
- **Deploy** — push to main, verify on Vercel or Railway
- **Gate** — check acceptance criteria, run E2E if applicable, only then proceed

Each tier ends with a tagged git release. If any slice fails its gate, we fix before moving on.

Tier 1 — Walking Skeleton

Week 1

Goal. End-to-end "hello world" with real infrastructure. Database → server → UI → deploy, all working, even if trivial.

Why this tier exists. Most projects collapse at week six because deployment was bolted on late. We prove the full path works on day seven before adding any product features.

Slice	What ships	Definition of done
1.1	Repo + CI/CD + blank deploy. Turborepo monorepo (apps/web, apps/pipeline, packages/ui, packages/db, packages/shared), GitHub repo, Vercel connected, GitHub Actions running typecheck and lint, blank Next.js page live at a public URL.	Push to main → green CI → live URL updates in under 90 seconds.
1.2	CLAUDE.md + Spec Kit + design folder. Constitution written, Spec Kit installed (uv tool install specify-cli, specify init . --integration claude), .specify/ and .claude/commands/ populated, six design PDFs in design/, design brief in docs/.	/speckit.constitution recognized in Claude Code; PDFs visible to Claude when referenced.
1.3	Design tokens + canonical Problem Card. Tailwind v4 with all Bristle color tokens (Editorial Light + Dark), Inter / Source Serif Pro / JetBrains Mono via next/font/google, lucide icons, theme switcher via next-themes, the canonical ProblemCardFull component built and rendered on the homepage with hardcoded data.	Card matches Core_app.pdf page 1 problem card pixel-for-4px-tolerance in both themes.
1.4	Supabase + Drizzle + one persisted problem. Supabase project created, pgvector extension enabled, Drizzle ORM configured, problems table with minimal schema, one seed problem inserted via migration, homepage fetches and renders it from the database.	Visit homepage → see real DB record rendered as a Problem Card → confirmed via Supabase dashboard.

Tier 1 release: v0.1.0 — walking-skeleton

Tier 2 — Public Surface

Weeks 2–3

Goal. Marketing site complete and indexable. Newsletter capture live. You can start collecting waitlist signups while the app is still being built.

Why now. Public pages don't depend on auth or pipeline. Building them now means a public URL exists for distribution before any feature work locks you indoors.

Slice	What ships	Definition of done
2.1	Landing page. Full landing from Public_pages.pdf page 1: hero with live Problem Card preview, "Find real problems worth solving" headline, source logo strip, "A research journal that doesn't sleep" three-card section, sample reports row, dark pricing teaser, full footer with status indicator.	Matches PDF, Lighthouse 90+, mobile responsive.
2.2	Pricing + FAQ. Pricing page with monthly/annual toggle, three tier cards with Pro highlighted, "Compare in detail" table, Enterprise card. FAQ with sticky left rail and accordion.	Both pages match PDFs; pricing toggle works; FAQ scroll-spy works.
2.3	About + Contact + Legal. About editorial layout with founder card and inline newsletter form. Contact page with three paths + form. Terms, Privacy, Security pages using the legal template with sticky TOC.	All four pages match PDFs; contact form sends via Resend; legal scroll-spy works.
2.4	Blog index + post + first article. Blog index "Field Notes" with filter chips and featured-article layout. Blog post template with right-rail TOC, pull-quote, inline figure, "Try Bristle" CTA card. One real published article ("What 50,000 GitHub issues reveal about developer pain").	Index and post match PDFs; article reads end-to-end; OG image renders correctly when shared.
2.5	Changelog. Public changelog with left jump-nav, dated entries grouped by month, feature/fix/improvement tags. RSS feed at /changelog.atom.	Matches PDF; RSS validates; month jump-nav works.
2.6	Public sample problem report. Public route /problems/[slug] reusing the authenticated problem detail layout, with sticky "viewing a free sample" banner and soft blur-overlay revealing only 5 of N quotes. Backed by real DB records, not static.	Visit a sample URL → see live data → blur reveal works → sign-up CTA functional.

Slice	What ships	Definition of done
2.7	Newsletter + sitemap + status integration. Newsletter signup wired to Resend audiences (or Beehiiv). sitemap.xml and robots.txt generated. Footer pulls status from Better Stack or equivalent showing "v1.0.0 · status: operational".	Submit newsletter form → confirmed in Resend; sitemap validates; status indicator updates if you flip a switch.

Tier 2 release: v0.2.0 — public-surface-live. Start collecting waitlist signups [here](#).

Tier 3 — Auth and Onboarding

Week 4

Goal. Users can sign up, verify, log in, recover passwords, and complete onboarding. Protected dashboard stub exists.

SSO deferred to v1.1. OAuth (Google + GitHub) ships now.

Slice	What ships	Definition of done
3.1	Sign-in + sign-up step 1 + protected route. Supabase Auth enabled with email/password. Sign-in page (Authentication.pdf page 2) and sign-up step 1 (Authentication.pdf page 1) with split-screen editorial layout, password strength meter, terms checkbox. Protected /app route shows "Welcome [name]" placeholder.	Sign up with email → land on /app. Sign out. Sign back in.
3.2	Sign-up step 2 + email OTP verification. "CREATE ACCOUNT · 2 OF 2" step capturing workspace name and initial preferences. Email verification page (Authentication.pdf page 5) with 6-digit OTP input, 30s resend cooldown, dedicated verification_codes table with expiry and rate limiting.	Sign up → receive code → enter → land on onboarding.
3.3	Password recovery. Forgot password (Authentication.pdf page 3) → reset password (Authentication.pdf page 4) with passphrase strength meter and rules list.	Request reset → receive email → set new password → sign in successfully.
3.4	OAuth Google + GitHub + callback bridge. OAuth providers configured in Supabase. Sign-in and sign-up pages include OAuth buttons. Callback bridge (Authentication.pdf page 6) with "Signing you in with Google" status card showing token exchange progress states.	OAuth flow completes end-to-end with both providers; callback bridge renders for at least 400ms.
3.5	Onboarding step 1 (role). /onboard/role with 3x2 grid of role cards (Indie founder, Product manager, Agency, Innovation lab, Researcher, Something else), progress dots "Step 1 of 3", "Skip for now" link. Persists user_role.	Choose role → continue → role saved → on step 2.

Slice	What ships	Definition of done
3.6	Onboarding step 2 (categories) with tier gating. /onboard/categories with flowing grid of category pills, each showing live problem count and sparkline. Search input. Footer banner enforcing minimum selection: "3 of 5 selected — pick 2 more to unlock instant alerts" on Starter, "3 of unlimited selected" on Pro. Persists user_categories.	Select required minimum → Continue enabled → categories saved.
3.7	First-run tour. Spotlight overlay on dashboard explaining: command palette (Cmd+K), problem cards, save action, alerts. Five tour steps with bubble cards (onboarding.pdf page 3). Skip and "End tour" options. Persists tour_completed flag.	New user lands on dashboard → tour runs → can be skipped → state persists across sessions.

Tier 3 release: v0.3.0 — auth-and-onboarding

Tier 4 — Authenticated App with Fixtures

Weeks 5–7

Goal. Entire product experience built against hand-crafted fixture data. UI feels complete; data isn't real yet. **This tier is critical** — it decouples UI development from pipeline development.

Approach. Write 15 high-quality problem fixtures as JSON, matching exactly the schema the pipeline will produce. UI reads from DB seeded with fixtures. When the pipeline goes live in Tier 5, no UI changes are required.

Slice	What ships	Definition of done
4.1	Full schema + 15 fixture problems seeded. Drizzle schema for: problems, problem_quotes, problem_sources, existing_solutions, wtp_signals, categories, user_saved_problems, saved_collections, alert_rules, alert_notifications, problem_activity_log, user_categories, usage_meters. 15 fixtures written covering all categories.	Schema migrated; fixtures visible in Supabase; Drizzle queries return typed results.
4.2	Dashboard with KPI cards + grid. Dashboard (Core_app.pdf page 1): "Good morning, [name]" with date label, four KPI cards (New mentions / 24h, Crossed momentum threshold, Saved problems, Alert queue) with computed values, sort tabs (Momentum / Frequency / Newest / WTP), 6-card problem grid, weekly momentum line chart, recent activity right rail.	All metrics compute from DB; sort works; grid renders six problem cards; chart renders with real fixture data.
4.3	Problem detail with 7 tabs. Problem detail (Core_app.pdf page 2): breadcrumb, title with "Validated demand" chip, momentum delta with sparkline, sources count and WTP mentions count. Seven tabs: Synthesis, Frequency, Evidence, Solutions, WTP, Related, Activity. Each tab fully populated. Sources donut, WTP panel, Who's complaining breakdown, Related problems list, all functional.	All 7 tabs render with fixture data; tab switching keyboard-navigable; deep links to tabs work (?tab=evidence).
4.4	Library with filters + momentum buckets. Library (Core_app.pdf page 3): left filter rail with momentum buckets (+100%, +25 to +99, Flat, New <30d), source checkboxes with counts, signal toggles, search input, table with momentum/sources/updated columns, pagination.	Filters compose with AND logic; URL state syncs; pagination works; counts update live.

Slice	What ships	Definition of done
4.5	Saved problems Kanban. Saved (Core_app.pdf page 4): four colored collections, drag-and-drop between columns (using @dnd-kit/core), "+ Add problem" inline, "New collection" CTA, upgrade banner ("Upgrade for unlimited"), per-collection count and color.	Drag works; collections persist; new collection flow works; tier limit enforced on Starter.
4.6	Alerts list + watch rules + delivery config. Alerts (Core_app.pdf page 5): feed grouped by date with tags (Momentum/New/WTP/Digest/Threshold/Weekly), right rail with active watch rules (toggleable) and delivery channels card. "New rule" CTA opens config modal. No real delivery yet — UI only.	Create rule → toggle channels → save → rule appears in active list; mock notifications render in feed.
4.7	Compare view with hardcoded scorecards. Compare (Core_app.pdf page 6): drag up to 4 problems into grid, qualitative scorecards (Validated demand, Has direct solution, Persona fit, Build effort, Defensibility), "Best fit" callout on top column, "Bristle's Read" row with categorical labels (STRONGEST / BUILD-ABLE / WATCH / SKIP). Scorecards and Read are hardcoded per-problem in fixtures for v1.0.	Drag works; 4-column max enforced; scorecards render; export to PDF works.
4.8	Command palette. Cmd+K opens centered overlay (Core_app.pdf page 7), grouped results (Problems, Categories, Actions), keyboard navigation (Up/Down/Enter/Esc), action shortcuts (S save, C compare, \ open in compare). Built with cmdk library.	Cmd+K from any page; type "stripe" → see problems; Enter navigates; Esc closes.
4.9	Add custom category. "+ Add category" button on dashboard opens modal: name, optional keywords, target sources checkboxes. Creates category in categories table with is_custom=true and created_by_user_id. UI immediately picks it up; pipeline picks it up in Tier 5.	Add category → appears in sidebar → can be selected as alert filter.

Tier 4 release: v0.4.0 — app-on-fixtures. You could demo this to investors or friends right now — it looks complete.

Tier 5 — Pipeline + Live Data

Weeks 8–11

Goal. Replace fixtures with real data flowing from Hacker News, GitHub Issues, and Stack Overflow. The pipeline is the most engineering-heavy part of the build. We don't rush it.

Slice	What ships	Definition of done
5.1	Python pipeline scaffold + HN ingester. apps/pipeline/ with FastAPI, Inngest for orchestration, deployed on Railway. Hacker News ingester via Algolia API runs every 4 hours, writes to raw_items with content hash dedup.	Job runs autonomously; raw_items table populating; no duplicates.
5.2	Filter classifier + embedder. LLM wrapper class with caching, audit logging, daily spend cap. Claude Haiku classifies each raw_item as complaint / feature-request / wish / bug / noise. OpenAI text-embedding-3-small generates vectors stored in pgvector.	80%+ noise filtered out; embeddings indexed in pgvector; per-call cost under \$0.0001 average.
5.3	Incremental clustering. HDBSCAN clustering on rolling 90-day window of embedded items. New items joined to existing clusters via nearest-neighbor on cosine similarity > 0.92, otherwise seed new cluster. Runs every 6 hours.	Clusters stable across runs; new HN posts join existing clusters when semantically similar; spot-check 10 clusters → all coherent.
5.4	Enricher + synthesizer. Per cluster, Claude Sonnet extracts title candidate, sentiment intensity, WTP quotes with stated prices, existing solutions mentioned. Synthesizer produces the 3-paragraph synthesis matching the editorial style of fixtures.	First 20 real problems produced; quality bar: you'd pay \$29/mo for these; spot-check synthesis matches voice of fixtures.
5.5	Fixtures → live data swap. Dashboard, library, problem detail, saved, compare all switch from fixture queries to live queries against real problems table. Fixture seed data preserved as test data.	UI unchanged from Tier 4; data is now live; momentum scores computed from real frequency over time.
5.6	GitHub Issues ingester. Curated list of 2,000 repos (top-starred in target categories) maintained in tracked_repos table. GraphQL API pulls issues with filters (closed-as-wontfix, stale, high comment count). Runs every 6 hours.	GitHub source visible in problem reports; cross-source clusters appear (HN + GitHub on same problem).

Slice	What ships	Definition of done
5.7	Stack Overflow ingester. Stack Exchange API pulls high-view questions with no accepted answer, filtered by tags relevant to categories. Runs daily.	Three-source clusters appearing; source distribution donut shows all three.
5.8	Cost controls + admin dashboard + daily summary. Admin route at /admin (gated to your email): per-source ingestion stats, processing throughput, daily LLM spend, quality metrics. Daily email at 9am with cost summary, new problems generated, anomalies.	You receive the email daily; cost stays under \$5/day at current scale; can drill into any anomaly.
5.9	Momentum buckets + cached aggregates. Daily job computes momentum score (% change over 14d vs prior 14d) and bucket (+100%, +25 to +99, Flat, New) for every problem. Cached aggregates: problems-per-source, problems-per-category, library filter counts.	Library filters fast (under 200ms); momentum chips accurate; sparklines render from cached 14-day series.
5.10	Quality tuning pass. Two weeks of pipeline output reviewed manually. Prompt engineering iterations on synthesizer. Manual override capability in admin: edit synthesis, merge clusters, mark cluster as noise.	Quality bar set by founder: would you pay for this? 80%+ of generated problems pass. Manual override used on 10–20%.

Tier 5 release: v0.5.0 — live-data. First moment the product is genuinely useful.

Tier 6 — Settings, Billing, Real Delivery

Weeks 12–13

Goal. Production-grade account management, payments captured, real notification delivery, API usable by external developers.

Per-seat billing deferred to v1.1. Team tier ships with fixed 5 seats; overage handled manually.

Slice	What ships	Definition of done
6.1	Profile page. Settings/Profile (User_and_Admin.pdf page 1): avatar upload to Supabase Storage, name/handle with availability check, primary email with verified badge, "Add backup email", role selector, timezone, date format, delete account flow with confirmation.	All fields persist; handle availability is real-time; theme persisted across devices.
6.2	Notifications matrix + quiet hours. Settings/Notifications (User_and_Admin.pdf page 3): delivery channels card (Email, In-app, Slack, Webhook) with verification states, 9-row × 4-column notification matrix, quiet hours with day picker.	Toggles persist; quiet hours suppress non-critical notifications; verification state correct after Slack OAuth.
6.3	Email digest + alert delivery (real). Daily digest job at 9am sends to Email channel. Momentum threshold alerts fire in real-time. All wired through Resend with templates: Welcome, Verification, Reset, Daily digest, Weekly digest, Momentum alert, Payment receipt.	Receive all email types; templates match Bristle voice; unsubscribe link works.
6.4	Lemon Squeezy checkout + tier gating. Checkout for Starter (\$29), Pro (\$79), Team (\$199) via Lemon Squeezy. Webhook handler updates subscriptions table. Feature gating throughout app: category limits, saved problems limits, API access, comparison limit, alert delivery channels.	Upgrade flow works end-to-end; webhook updates within 5s; tier limits enforced in code paths (not just UI).
6.5	Billing page + invoice history + usage meters. Settings/Billing (User_and_Admin.pdf page 2): current plan with usage stat cards (Categories, Saved problems, API calls this month, Alerts this month), each with fill bar showing % of quota. Payment method, upcoming invoice, invoice history table with download.	Usage meters accurate; invoices download as PDF; switch plan flow works (proration handled by Lemon Squeezy).

Slice	What ships	Definition of done
6.6	API keys + rate limiting + usage tracking. Settings/API Keys (User_and_Admin.pdf page 4): usage chart by call type (Reports, Search, Webhooks), keys table with scopes (Read / Read+Write), example curl with syntax highlighting. Public API at api.bristle.dev/v1/problems with bearer auth, rate limited at 50k/mo Pro, 200k/mo Team, usage logged per request.	API returns expected JSON for fixture queries; rate limit headers correct; usage chart updates within minutes.
6.7	Team page + roles + invite (no SSO). Settings/Team (User_and_Admin.pdf page 5): seat usage bar, members table with four roles (Owner / Admin / Editor / Viewer), invite by link and email, roles & permissions reference card. RBAC enforced server-side.	Invite flow works; roles enforced (Editor can't change billing; Viewer can't save); 5-seat limit hard-enforced.
6.8	Integrations: Slack + Webhooks + Zapier. Settings/Integrations (User_and_Admin.pdf page 6) showing all 10 partners but only 3 functional: Slack OAuth for alert delivery, Webhook with secret signing, Zapier via REST API trigger. Other 7 show "Coming v1.1" or "Connect" that opens a waitlist.	Slack delivers alerts to a channel; webhook signs payloads; Zapier trigger works in their builder.

Tier 6 release: v0.6.0 — production-billing-and-delivery

Tier 7 — System Pages + Polish

Week 13

Goal. Everything between "works for me" and "ready for paying strangers."

Slice	What ships	Definition of done
7.1	404 with trending list. 404 (System_pages.pdf page 1): "We searched 142,318 problems. None of them were this page" headline, "Trending while you're here" list with three live problems, command palette CTA, SIGNAL LOST decorative chart.	Visiting any bad URL shows 404 with live trending problems.
7.2	500 with incident integration. 500 (System_pages.pdf page 2): incident card with request_id and timestamps, integrated with Better Stack (or Statuspage) for live incident status. "Try again" reloads.	Trigger a forced 500 → page renders; incident card pulls from incident provider API.
7.3	Maintenance mode. Maintenance (System_pages.pdf page 3): admin-toggleable. Static log component (live log deferred to v1.1). Auto-refreshes every 60s.	Toggle maintenance flag → all routes show maintenance page except /admin and /status.
7.4	Success states with receipts. Success state (System_pages.pdf page 4): "Welcome to Pro, [name]" with receipt block pulling from Lemon Squeezy webhook payload, "What's next" checklist with three action items.	Complete Pro upgrade → land on success state → receipt accurate → checklist actions work.
7.5	Indexing state for new categories. When user adds a custom category, the indexing state (System_pages.pdf page 5) shows "The signal is warming up" with per-source progress bars and "While you wait" panel suggesting adjacent categories. Real progress from pipeline.	Add custom category → see indexing page → adjacent categories load → eventually shows first problems.
7.6	Loading skeletons across app. All async views replaced with skeleton patterns matching System_pages.pdf page 6. Skeleton with timing tooltip ("Crunching 4,318 mentions... Usually ~1.4s · taking 3.8s") on slow queries (>2s).	Throttle network → see skeletons → see timing tooltip on slow queries.

Slice	What ships	Definition of done
7.7	E2E test suite. Playwright tests for critical paths: signup → onboard → save → alert → upgrade → API call → cancel. Runs on every PR via GitHub Actions.	Suite green on main; tests run in under 5 minutes.
7.8	Lighthouse + accessibility + mobile pass. Lighthouse 90+ on landing, dashboard, problem detail. WCAG 2.2 AA audit fixes (focus rings, aria-labels, contrast, keyboard nav). Mobile responsive verified at 375/768/1024 breakpoints.	All scores hit; manual screen reader pass clean; mobile flows work end-to-end.
7.9	Error monitoring + product analytics. Sentry wired for errors with source maps. PostHog wired for product events (signup, save, alert, upgrade) and session replay.	Trigger error → see in Sentry within 30s; check funnel in PostHog.

Tier 7 release: v0.7.0 — launch-ready

Tier 8 — Pre-Launch + Launch

Week 14

Goal. Public launch with coordinated distribution.

Slice	What ships	Definition of done
8.1	200 sample problem reports indexed for SEO. 200 curated problem reports made public at /problems/[slug]. Each has schema.org markup, OG image, sitemap entry. Internal links from blog post to relevant problems.	Google Search Console shows 200 indexed within a week of submission.
8.2	Newsletter ready + first 4 issues drafted. Beehiiv (or Resend) configured. First 4 Field Notes issues drafted: launch issue, "5 problems with momentum > 200%", "What HN complained about last week", a deeper data analysis post.	First issue scheduled to send on launch day +1.
8.3	Lifetime deal landing. /lifetime page with countdown, 200-spot limit, \$99 one-time for Pro lifetime. Lemon Squeezy product configured.	Page live, checkout works, counter is real.
8.4	Launch coordination assets. Product Hunt page drafted (gallery images, tagline, description), HN "Show HN" post drafted, IndieHackers post drafted, Twitter launch thread drafted (with 60s screen recording per major feature).	All assets reviewed by two friends; thread scheduled for 8am PST launch day.
8.5	Launch day. Product Hunt goes live at 12:01am PST. HN post at 8am PST. IndieHackers post at 9am. Twitter thread at 9:30am. Email to waitlist at 10am. You available for 12 hours of support responsiveness.	100 free signups within 48 hours; first paying customer within 7 days; HN front page (stretch goal).

Tier 8 release: v1.0.0 — public-launch

Explicit Deferrals (v1.1 or later)

These appear in the design PDFs but are deliberately out of scope for v1.0:

- **SSO (SAML/Enterprise)** — UI shows SSO button but it's hidden behind a feature flag.
- **Per-seat overage billing** — Team tier ships with hard 5-seat limit.
- **312 categories** — v1 ships with ~40 curated; users request more via "Request category" button.
- **"Bristle's Read" LLM** — hardcoded in fixtures and never replaced for v1.0; goes live in v1.1 once we have real comparison data.
- **MS Teams, Discord, Linear, Notion, n8n, CSV export, direct REST integration** — 7 of 10 integrations show "Coming v1.1" or join-waitlist.
- **Live migration log streamer** on maintenance page.
- **Reddit ingestion** — platform risk too high.
- **Mobile app** — web responsive only.
- **Real-time updates via WebSocket** — all updates are batch (4–6h cadence).

Working Agreement

For every slice, the rhythm is:

1. The slice prompt is provided to run `/speckit.specify` in Claude Code.
2. Paste it, Claude generates the spec.
3. Share the spec, iterate until tight.
4. Run `/speckit.plan` with the don't-implement-yet guard phrase.
5. Share the plan, iterate.
6. Run `/speckit.tasks`, then `/speckit.implement`.
7. Commit, push, deploy, verify.
8. Report what worked or what broke.
9. Only then start the next slice.

You don't move to the next slice without confirmation that the current one is done. No next prompt is given without that confirmation. That's how we don't get stuck.

End of Build Plan — Bristle v1.0